ExpressionSet

Kasper D. Hansen

Dependencies

This document has the following dependencies:

```
library(Biobase)
library(ALL)
library(hgu95av2.db)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("Biobase", "ALL", "hgu95av2.db"))
```

Corrections

Improvements and corrections to this document can be submitted on its GitHub in its repository.

Overview

We will examine how to use and manipulate the `ExpressionSet` class; a fundamental data container in Bioconductor. The class is defined in the *Biobase* package.

This class has inspired many other data containers in Bioconductor and can be considered a great success in programming with data.

Other Resources

- The "An Introduction to Bioconductor's ExpressionSet Class" vignette from the Biobase webpage.
- The "Notes for eSet Developers" vignette from the Biobase webpage contains advanced information.

Start

An example dataset, stored as an `ExpressionSet` is available in the *ALL* package. This package is an example of an "experimental data" package"; a bundling of a full experiment inside an R package. These experimental data packages are used for teaching, testing and illustration, but can also serve as the documentation of a data analysis.

```
library(ALL)
data(ALL)
ALL
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 12625 features, 128 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: 01005 01010 ... LAL4 (128 total)
##   varLabels: cod diagnosis ... date last seen (21 total)
##   varMetadata: labelDescription
## featureData: none
## experimentData: use 'experimentData(object)'
##   pubMedIds: 14684422 16243790
## Annotation: hgu95av2
```

(you don't always have to explicitly call `data()` on all datasets in R; that depends on the choice of the package author).

This is an experiment done on an Affymetrix HGU 95av2 gene expression microarray; the authors profiled 128 samples.

Because this is data from an experiment package, there is documentation in the help page for the dataset, see

```
?ALL
```

From the printed description of `ALL` you can see that 12625 features (in this case genes) were measured on 128 samples. The object contains information about the experiment; look at

```
experimentData(ALL)
```

```
## Experiment data
##   Experimenter name: Chiaretti et al.
##   Laboratory: Department of Medical Oncology, Dana-Farber Cancer Inst
##   Contact information:
##   Title: Gene expression profile of adult T-cell acute lymphocytic le
##   URL:
##   PMIDs: 14684422 16243790
##
##   Abstract: A 187 word abstract is available. Use 'abstract' method.
```

Two papers (pubmed IDs or PMIDs) are associated with the data.

There is no `protocolData` in the object (this is intended to describe the experimental protocol); this is typical in my experience (although it would be great to have this information easily available to the end user).

The core of the object are two matrices

- the `exprs` matrix containing the 12625 gene expression measurements on the 128 samples (a 12625 by 128 numeric matrix).
- the `pData` `data.frame` containing phenotype data on the samples.

You get the expression data by

```
exprs(ALL)[1:4, 1:4]
```

```
##              01005    01010    03002    04006
## 1000_at   7.597323 7.479445 7.567593 7.384684
## 1001_at   5.046194 4.932537 4.799294 4.922627
## 1002_f_at 3.900466 4.208155 3.886169 4.206798
## 1003_s_at 5.903856 6.169024 5.860459 6.116890
```

Note how this matrix has column and rownames. These are `sampleNames` and `featureNames`. Get them by

```
head(sampleNames(ALL))
```

```
## [1] "01005" "01010" "03002" "04006" "04007" "04008"
```

```
head(featureNames(ALL))
```

```
## [1] "1000_at"   "1001_at"   "1002_f_at" "1003_s_at" "1004_at"   "1005
```

To get at the `pData` information, using the `pData` accessor.

```
head(pData(ALL))
```

```
##        cod diagnosis sex age BT remission CR   date.cr t(4;11) t(9;22
## 01005 1005 5/21/1997   M  53 B2        CR CR  8/6/1997   FALSE    TRU
## 01010 1010 3/29/2000   M  19 B2        CR CR 6/27/2000   FALSE   FALS
## 03002 3002 6/24/1998   F  52 B4        CR CR 8/17/1998      NA      N
## 04006 4006 7/17/1997   M  38 B1        CR CR  9/8/1997    TRUE   FALS
## 04007 4007 7/22/1997   M  57 B2        CR CR 9/17/1997   FALSE   FALS
## 04008 4008 7/30/1997   M  17 B1        CR CR 9/27/1997   FALSE   FALS
##       cyto.normal        citog mol.biol fusion protein mdr   kinet
## 01005       FALSE      t(9;22)  BCR/ABL           p210 NEG dyploid FA
## 01010       FALSE  simple alt.      NEG           <NA> POS dyploid FA
## 03002          NA         <NA>  BCR/ABL           p190 NEG dyploid FA
## 04006       FALSE      t(4;11) ALL1/AF4           <NA> NEG dyploid FA
## 04007       FALSE      del(6q)      NEG           <NA> NEG dyploid FA
## 04008       FALSE complex alt.      NEG           <NA> NEG hyperd. FA
##       relapse transplant               f.u date last seen
## 01005   FALSE       TRUE BMT / DEATH IN CR           <NA>
## 01010    TRUE      FALSE               REL      8/28/2000
## 03002    TRUE      FALSE               REL     10/15/1999
## 04006    TRUE      FALSE               REL      1/23/1998
## 04007    TRUE      FALSE               REL      11/4/1997
## 04008    TRUE      FALSE               REL     12/15/1997
```

You can access individual columns of this `data.frame` by using the `$` operator:

```
head(pData(ALL)$sex)
```

```
## [1] M M F M M M
## Levels: F M
```

```
head(ALL$sex)
```

```
## [1] M M F M M M
## Levels: F M
```

Subsetting

Subsetting of this object is an important operation. The subsetting has two dimensions; the first dimension is genes and the second is samples. It keeps track of how the expression measures are matched to the pheno data.

```
ALL[,1:5]
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 12625 features, 5 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: 01005 01010 ... 04007 (5 total)
##   varLabels: cod diagnosis ... date last seen (21 total)
##   varMetadata: labelDescription
## featureData: none
## experimentData: use 'experimentData(object)'
##   pubMedIds: 14684422 16243790
## Annotation: hgu95av2
```

```
ALL[1:10,]
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 10 features, 128 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: 01005 01010 ... LAL4 (128 total)
##   varLabels: cod diagnosis ... date last seen (21 total)
##   varMetadata: labelDescription
## featureData: none
## experimentData: use 'experimentData(object)'
##   pubMedIds: 14684422 16243790
## Annotation: hgu95av2
```

```
ALL[1:10,1:5]
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 10 features, 5 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: 01005 01010 ... 04007 (5 total)
##   varLabels: cod diagnosis ... date last seen (21 total)
##   varMetadata: labelDescription
## featureData: none
## experimentData: use 'experimentData(object)'
##   pubMedIds: 14684422 16243790
## Annotation: hgu95av2
```

We can even change the order of the samples

```
ALL[, c(3,2,1)]
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 12625 features, 3 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: 03002 01010 01005
##   varLabels: cod diagnosis ... date last seen (21 total)
##   varMetadata: labelDescription
## featureData: none
## experimentData: use 'experimentData(object)'
##   pubMedIds: 14684422 16243790
## Annotation: hgu95av2
```

```
ALL$sex[c(1,2,3)]
```

```
## [1] M M F
## Levels: F M
```

```
ALL[, c(3,2,1)]$sex
```

```
## [1] F M M
## Levels: F M
```

This gives us a lot of confidence that the data is properly matched to the phenotypes.

featureData and annotation

You can think of `pData` as providing information on the columns (samples) of the measurement matrix. Similar to `pData`, we have `featureData` which is meant to provide information on the features (genes) of the array.

However, while this slot has existed for many years, it often goes unused:

```
featureData(ALL)
```

```
## An object of class 'AnnotatedDataFrame': none
```

So this leaves us with a key question: so far the `ALL` object has some useful description of the type of experiment and which samples were profiled. But how do we get information on which genes were profiled?

For commercially available microarrays, the approach in Bioconductor has been to make so-called annotation packages which links `featureNames` to actual useful information. Conceptually, this information could have been stored in `featureData`, but isn't.

Example

```
ids <- featureNames(ALL)[1:5]
ids
```

```
## [1] "1000_at"   "1001_at"   "1002_f_at" "1003_s_at" "1004_at"
```

these are ids named by Affymetrix, the microarray vendor. We can look them up in the annotation package by

```
library(hgu95av2.db)
as.list(hgu95av2ENTREZID[ids])
```

```
## $`1000_at`
## [1] "5595"
##
## $`1001_at`
## [1] "7075"
##
## $`1002_f_at`
## [1] "1557"
##
## $`1003_s_at`
## [1] "643"
##
## $`1004_at`
## [1] "643"
```

This gives us the Entrez ID associated with the different measurements. There are a number of so-called "maps" like `hgu95av2XX` with `XX` having multiple values. This approach is very specific to Affymetrix chips. An alternative to using annotation packages is to use the ***biomaRt*** package to get the microarray annotation from Ensembl (an online database). This will be discussed elsewhere.

We will leave the annotation subject for now.

Note: phenoData and pData

For this type of object, there is a difference between `phenoData` (an object of class `AnnotatedDataFrame`) and `pData` (an object of class `data.frame`).

The idea behind `AnnotatedDataFrame` was to include additional information on what a `data.frame` contains, by having a list of descriptions called `varLabels`.

```
pD <- phenoData(ALL)
varLabels(pD)
```

```
##  [1] "cod"           "diagnosis"     "sex"           "age"
##  [5] "BT"            "remission"     "CR"            "date.cr"
##  [9] "t(4;11)"       "t(9;22)"       "cyto.normal"   "citog"
## [13] "mol.biol"      "fusion protein" "mdr"          "kinet"
## [17] "ccr"           "relapse"       "transplant"    "f.u"
## [21] "date last seen"
```

But these days, it seems that `varLabels` are constrained to be equal to the column names of the `data.frame` making the entire `AnnotatedDataFrame` construction unnecessary:

```
varLabels(pD)[2] <- "Age at diagnosis"
pD
```

```
## An object of class 'AnnotatedDataFrame'
##   sampleNames: 01005 01010 ... LAL4 (128 total)
##   varLabels: cod Age at diagnosis ... date last seen (21 total)
##   varMetadata: labelDescription
```

```
colnames(pD)[1:3]
```

```
## [1] "cod"              "Age at diagnosis" "sex"
```

```
varLabels(pD)[1:3]
```

```
## [1] "cod"              "Age at diagnosis" "sex"
```

So now we have exposed what is arguably a bug, together with some un-used abstraction. This happens.

The eSet class

The `ExpressionSet` class is an example of an `eSet`. The `ExpressionSet` class has a single measurement matrix which we access by `exprs`. In contrast to this, the `eSet` class can have any number of measurement matrices with arbitrary names, although all matrices needs to have the same dimension.

An example of another `eSet` derived class is `NChannelSet` which is meant to store multichannel microarray measurements; an example is a two-color array where the microarray has been imaged in two different colors.

Another example of classes are from the ***minfi*** package for DNA methylation arrays; here there are classes such as `RGChannelSet` for a two color microarray and `MethylSet` for methylation measurements.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
##  [1] hgu95av2.db_3.1.3    org.Hs.eg.db_3.1.2   RSQLite_1.0.0
##  [4] DBI_0.3.1            AnnotationDbi_1.30.1 GenomeInfoDb_1.4.2
##  [7] IRanges_2.2.7        S4Vectors_0.6.3      ALL_1.10.0
## [10] Biobase_2.28.0       BiocGenerics_0.14.0  BiocStyle_1.6.0
## [13] rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.8    formatR_1.2     magrittr_1.5    evaluate_0.7.2
##  [5] stringi_0.5-5   tools_3.2.1     stringr_1.0.0   yaml_2.1.13
##  [9] htmltools_0.2.6 knitr_1.11
```